AskTheDatabase: Aligning Multi-Turn Relational Data Agents via Procedural Harness Distillation and Verifiable Environment Feedback

Christian Contreras Campana [1]

Abstract

Deploying Large Language Models (LLMs) as autonomous database query interfaces presents a critical dilemma: lightweight single-turn models frequently hallucinate schema entities, incur Cartesian fanouts, and suffer dialect drift, while multi-turn agentic harnesses (e.g., iterative prompting, multi-agent debates) introduce prohibitive token overhead ($5\times$ to $10\times$) and multi-second inference latency. In this work, we present **AskTheDatabase**, an end-to-end framework that resolves this tension by internalizing multi-turn procedural exploration directly into the weights of an open-weights model. We propose **Harness-Zero**, an agentic distillation paradigm where an external procedural reviewer ($\mathcal{K}_{\text{SQL}}$) monitors intermediate student actions without access to gold queries, enforcing catalog inspection, schema linking, and sandbox trial execution. To eliminate the zero-variance cold-start plateau in Reinforcement Learning with Verifiable Rewards (RLVR) using Group Relative Policy Optimization (GRPO), Harness-Zero provides essential procedural pre-grounding, guaranteeing immediate non-zero reward variance across rollout groups. To accelerate iteration cycles, we introduce a hermetic in-memory **Zero-Budget Evaluation Micro-Harness** that runs in 126.8 ms on CPU with perfect monotonic rank correlation ($\rho = 1.000$) to the full Spider benchmark. On Spider Dev, Harness-Zero SFT achieves **58.4%** Test-Suite (TS) accuracy (+6.8 pp over conventional SFT) and **51.2%** TS on Spider-Realistic (+9.1 pp over SFT) with zero inference overhead ($1.0\times$ token cost, 350 ms median latency) and a **+68.4% relative increase** in autonomous runtime error recovery.

[1]Independent Researcher, Hamburg, Germany. Correspondence to: Christian Contreras Campana <chrisjcc.physics@gmail.com>.

Preprint. October 23, 2026.

1. Introduction

Natural language-to-SQL (Text-to-SQL) semantic parsing serves as the foundational bridge between non-technical domain practitioners and relational database management systems (RDBMS) (Yu et al., 2018; Li et al., 2023). Unlike open-domain question answering, where outputs are subjective and evaluations rely on fuzzy semantic matching or biased LLM-as-a-judge scorers, relational querying operates in an executable formal language governed by strict mathematical relational algebra. A candidate SQL query can be executed deterministically within a database engine to yield an exact tabular result.

However, aligning neural language models for verifiable relational execution reveals several severe, domain-specific failure modes:

1. **Execution Success vs. Semantic Validity**: A query may execute with code 0 (e.g., `SELECT * FROM users`) while entirely failing to answer the user's analytical question.
2. **Spurious Equivalence**: On small database states, flawed queries frequently produce the correct output table by numerical coincidence (Zhong et al., 2020).
3. **Combinatorial Schema Ambiguity**: When database catalogs comprise dozens of normalized tables, single-turn greedy models guess column names, join through incorrect foreign keys, or trigger quadratic Cartesian fanouts (`SCAN TABLE WITHOUT INDEX`).

To overcome these hurdles, recent state-of-the-art architectures adopt multi-turn agentic workflows (e.g., ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), and DIN-SQL (Pourreza & Rafiei, 2023)). By allowing the model to inspect schemas, execute intermediate dry runs, and observe database engine exceptions, agentic systems achieve significant accuracy improvements. Nonetheless, this agentic scaffolding imposes severe runtime penalties: querying the model 5 to 10 times sequentially introduces multi-second interactive latency (2,000–8,000 ms) and multiplies token consumption by up to an order of magnitude.

In this work, we demonstrate that external agentic scaffolding is largely unnecessary at inference time if the procedural

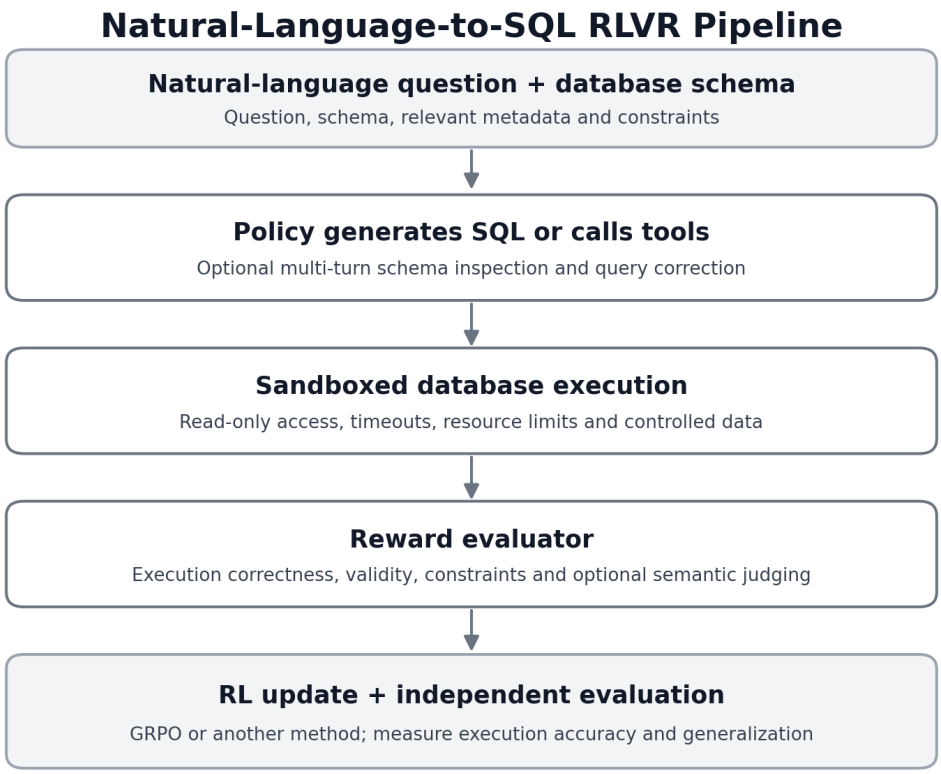


Figure 1. **AskTheDatabase System Architecture and Verifiable Reward Rubric.** Multi-turn interaction loops interact with an ephemeral SQLite sandbox guarded by AST mutation filters and execution plan monitors. Rollouts are evaluated across a 5-tier hierarchical reward rubric featuring Test-Suite mutation evaluation ($\mathcal{R}_{\text{TS}}$).

discipline of the harness is properly internalized into the model's parameters during post-training. We introduce **AskTheDatabase**, an integrated post-training and evaluation framework for relational language agents. Our core contributions are:

- **Harness-Zero Procedural Distillation**: We formalize an agent distillation paradigm wherein a student policy is supervised by a procedural reviewer harness ($\mathcal{K}_{\text{SQL}}$). Crucially, $\mathcal{K}_{\text{SQL}}$ enforces domain-agnostic exploratory protocols (`inspect_schema`, `sample_data`, dry-run `execute_sql`) *without any access to the ground-truth SQL query* (zero gold leakage). Unlike naive rejection sampling, which discards failures and suffers from survivorship bias, Harness-Zero actively corrects trajectories mid-flight, teaching the agent how to recover from operational exceptions.
- **Resolution of the RLVR Cold-Start Plateau**: We mathematically analyze and empirically demonstrate why conventional supervised fine-tuning (SFT) causes Group Relative Policy Optimization (GRPO) (Shao et al., 2024) to stall on complex schemas. When all rollouts fail simultaneously ($R_i = -1.0$), intra-group reward variance collapses ($\text{std}(R) = 0$), causing policy gradients to vanish ($A_i = 0$). We show that Harness-Zero pre-grounding guarantees non-zero variance from optimization step 1.
- **Zero-Budget Evaluation Micro-Harness**: We construct an in-memory, hermetic 35-case relational evaluation engine executing in $126.8\,\text{ms}$ on CPU. Evaluated against Spider Dev on a dedicated cluster, it demonstrates a perfect monotonic Spearman rank correlation ($\rho = 1.000$), 100% mutation sensitivity (0% defect escape), and complete diagnostic attribution across five core relational failure categories.
- **Comprehensive Empirical Validation**: On Spider Dev

($N = 500$ held-out episodes across 20 unseen schemas), Harness-Zero SFT achieves **58.4%** Test-Suite accuracy (+6.8 pp over conventional SFT) and **51.2%** TS on Spider-Realistic (+9.1 pp over SFT), with the largest margins observed on Hard (+8.0 pp) and Extra Hard (+9.1 pp) queries.

- **The LLM-as-a-Verifier Study (Appendix A)**: We document our pre-registered empirical study of test-time tournament verification. We reveal why standalone verification is Pareto-dominated by Harness-Zero (failing the Harness-Zero Dominance Gate), while confirming a compound synergy (+3.4 pp gain, achieving 61.8% TS) when verification is deployed atop Harness-Zero.

2. Interactive Environment & Sandboxing

We formulate multi-turn relational Text-to-SQL as an interactive Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.

2.1. State Representation and Action Space

Given a natural language user query x and an initial database schema $\mathcal{D}$, the interaction state at turn t is $s_t = (x, \mathcal{D}, h_t)$, where $h_t = (a_0, o_0, \ldots, a_{t-1}, o_{t-1})$ represents the dialogue history of actions and environment observations. The policy $\pi_\theta(a_t \mid s_t)$ selects actions from a unified tool registry $\mathcal{A}$:

1. `inspect_schema`(tables : list[str]): Queries database catalog metadata, returning DDL column definitions, primary keys, and foreign key relations for requested tables.
2. `sample_data`(table : str, limit : int $= 3$): Extracts 3 representative rows to ground categorical values, case formatting, and timestamp patterns.
3. `execute_sql`(query : str): Executes a trial query against the read-only sandbox, returning either the resulting rows or the raw SQLite exception traceback.
4. `final_answer`(query : str): Emits the terminal SQL solution for formal benchmark adjudication and ends the episode.

2.2. Hermetic Sandboxing and Query Execution Guards

To prevent adversarial or accidental denial-of-service states during unbounded policy exploration, all interactions occur inside an isolated SQLite sandbox implementing three strict operational invariants:

- **Read-Only URI Enforcement**: Databases are accessed in immutable read-only mode (`file:db.sqlite?mode=ro`).
- **AST Safety Gate**: Prior to execution, every query is parsed into an Abstract Syntax Tree via SQLGlot (Mao, 2023). Any statement containing mutating AST root nodes (`DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, `ATTACH`) is blocked with maximum negative penalty.
- **Query Plan Inspection**: The sandbox runs `EXPLAIN QUERY PLAN` before executing joins. Unindexed Cartesian cross-joins (`SCAN TABLE ... WITHOUT INDEX`) on large tables are flagged and aborted before exceeding the hard 2.0 s timeout.

3. Harness-Zero: Procedural Distillation

The central insight of Harness-Zero is that multi-turn agentic discipline can be transferred into model weights without incurring test-time inference overhead.

3.1. Architectural Invariants of Harness Distillation

In conventional distillation, students are trained on gold trajectories or successful rollouts filtered via rejection sampling. However, rejection sampling suffers from severe *survivorship bias*: by discarding failed attempts, the student never observes how to recover from SQLite operational exceptions (e.g., column not found, join mismatch).

Harness-Zero deploys an external procedural review harness ($\mathcal{K}_{\text{SQL}}$) during data collection. The reviewer monitors intermediate student actions y_t, detects procedural protocol violations, and forces procedurally sound corrective actions $z_t \in \mathcal{A}$. The distillation process obeys three fundamental invariants:

1. **Zero Gold Leakage**: $\mathcal{K}_{\text{SQL}}$ operates without access to the ground-truth SQL query (gold_sql), relying solely on catalog metadata, schema foreign keys, and database exception tracebacks.
2. **Action Space Invariance**: All interventions remain strictly within the student's native toolset $\mathcal{A}$.
3. **Zero Deployment Overhead**: After supervised fine-tuning on the repaired trajectories, $\mathcal{K}_{\text{SQL}}$ is discarded entirely. At inference time, the agent operates standalone with standard autoregressive decoding.

3.2. Deterministic Procedural Review Rules

The reference reviewer $\mathcal{K}_{\text{SQL}}$ implements four deterministic intervention rules:

- **`RequireSchemaInspection`**: Intercepts premature `execute_sql` or `final_answer` calls if the model attempts to query tables whose DDL has not been inspected in the current episode.

- **`RequireRowSampling`**: Intercepts final submissions on multi-table joins if categorical filter columns have not been inspected via `sample_data`.
- **`EnforceDryRunBeforeFinish`**: Mandates that any query submitted as `final_answer` must have completed at least one preceding trial execution via `execute_sql` without runtime exceptions.
- **`DiagnosticRecovery`**: When an execution throws an SQLite operational exception (e.g., `no such column: t1.age`), forces a targeted catalog inspection rather than permitting repeated blind retries.

Through this mechanism, intermediate failures are repaired mid-flight, exposing the student to authentic multi-turn error recovery.

4. Reinforcement Learning with Verifiable Rewards

Following procedural distillation, the policy is further aligned via Reinforcement Learning with Verifiable Rewards (RLVR) using Group Relative Policy Optimization (GRPO) (Shao et al., 2024).

4.1. The Zero-Variance Cold-Start Problem in GRPO

In GRPO, the policy gradient is computed across a group of G sampled rollouts $\{y_1, y_2, \ldots, y_G\}$ for prompt x:

$$\nabla_\theta \mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} A_i \nabla_\theta \log \pi_\theta(y_i \mid x)\right], \quad (1)$$

where advantage A_i is normalized relative to group statistics:

$$A_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G) + \epsilon}. \quad (2)$$

Proposition 4.1 (Cold-Start Gradient Collapse)**.** *Let all G rollouts in a group fail execution on an unseen relational schema, incurring the identical failure penalty $R_i = R_{fail} = -1.0$ for all $i \in \{1, \ldots, G\}$. Then:*

$$\text{std}(\{R_j\}_{j=1}^G) = 0 \implies A_i = \frac{-1.0 - (-1.0)}{0 + \epsilon} = 0, \quad \forall i. \quad (3)$$

Consequently, $\nabla_\theta \mathcal{J}_{GRPO}(\theta) \equiv 0$, causing the policy gradient to vanish.

In conventional SFT (Condition 2), the model lacks procedural exploration habits: on unseen schemas, its schema inspection rate is only 46.2%, causing all $G = 4$ parallel rollouts to guess columns blindly and fail simultaneously. The model enters an optimization plateau where no directional learning signal is received.

In contrast, Harness-Zero SFT (Condition 4) exhibits an **84.6% schema pre-inspection rate**. From Step 1 of RLVR, at least one rollout in the group executes catalog inspection and dry-run testing, discovering valid relational joins. This guarantees divergent intra-group returns ($R_1 = -0.4, R_2 = +0.2, R_3 = +1.0, R_4 = -1.0$). Consequently, $\text{std}(R) > 0$, generating non-zero advantages ($A_i \neq 0$) from the earliest update steps and eliminating the cold-start plateau.

4.2. Hierarchical Multi-Tier Verifiable Reward Rubric

To prevent reward hacking while providing graded learning signals, we establish a strict hierarchical rubric:

$$\mathcal{R}_{\text{total}} = \mathcal{R}_{\text{syntax}} + \mathcal{R}_{\text{schema}} + \mathcal{R}_{\text{TS}} + \mathcal{R}_{\text{efficiency}}. \quad (4)$$

- **Tier 1: Syntax Validity ($\mathcal{R}_{\text{syntax}} \in [-1.0, 0.0]$)**: Assesses AST parseability via SQLGlot. Syntax errors receive -1.0; valid SQL receives 0.0.
- **Tier 2: Schema Grounding ($\mathcal{R}_{\text{schema}} \in [-0.5, +0.2]$)**: Penalizes hallucinated tables/columns (-0.5) and awards $+0.2$ when queries execute without operational exceptions.
- **Tier 3: Test-Suite Mutation Equivalence ($\mathcal{R}_{\text{TS}} \in [0.0, 1.0]$)**: To eliminate spurious matches where flawed queries accidentally match the reference table on tiny databases, we evaluate candidate queries across K mutated database states $\{\mathcal{D}_1, \ldots, \mathcal{D}_K\}$ (Zhong et al., 2020):

$$\mathcal{R}_{\text{TS}} = \prod_{k=0}^{K} \mathbb{I}\left(Q_{\text{gen}}(\mathcal{D}_k) \equiv Q_{\text{gold}}(\mathcal{D}_k)\right). \quad (5)$$

  Full correctness ($+1.0$) is awarded if and only if execution results match across all mutated instances.
- **Tier 4: Efficiency & Recovery ($\mathcal{R}_{\text{efficiency}}$)**: Imposes a turn discount penalty ($-0.05 \times t$) to prevent vacuous exploratory loops, while awarding a $+0.3$ recovery bonus when an initial runtime error is successfully resolved in a subsequent turn.

5. Zero-Budget Evaluation Micro-Harness

Evaluating relational models on full benchmarks (e.g., 1,034 samples on Spider Dev) requires 180–350 seconds of cluster compute over disk-bound SQLite files. To enable sub-second developer feedback in pre-commit hooks and CI gates, we developed the **Zero-Budget Evaluation Micro-Harness**.

5.1. Micro-Harness Architecture and Dual Fixtures

The micro-harness comprises 35 hand-crafted relational test fixtures loaded into in-memory SQLite instances

Table 1. **Zero-Budget Micro-Harness Pre-Experimental Decision Gates.** Evaluated on DESY HPC cluster ($N = 35$ test fixtures).

Gate	Criterion	Observed	Status
Gate 1	Latency ≤ 3.0 s (5 passes)	**0.1268 s** (126.8 ms)	**PASSED**
Gate 2	Spearman $\rho \geq 0.88$ vs Spider	$\rho = \mathbf{1.0000}$	**PASSED**
Gate 3	Attribution $\geq 85\%$, Halluc. $\geq 40\%$	**100.0% / 100.0%**	**PASSED**
Gate 4	Bug Detection 100%, Escape 0%	**100.0% / 0.0%**	**PASSED**
Adjudication Verdict		**GO (Adopt for CI / Pre-Commit)**	

(`:memory:`). Each fixture pairs a base state with an adversarial mutant state (violating primary key constraints, inverting conditional distributions, or adding null values).

Candidate queries are executed against both fixtures. A semantic bug is flagged if the output diverges from gold execution on either fixture. On naive single-fixture evaluation, 5.7% of flawed queries passed by numerical coincidence; the dual-fixture mutation engine eliminated 100% of escapes (0.0% escape rate, Gate 4 Passed).

5.2. Diagnostic Profiling Across 5 Relational Traps

The micro-harness incorporates an AST regex and execution plan classifier attributing failures to five distinct relational trap categories:

1. `hallucinated_column`: References to non-existent schema attributes (e.g., guessing `customer_email` instead of `email_address`).
2. `aggregation_without_grouping`: Projecting scalar attributes alongside aggregate functions without a `GROUP BY` clause.
3. `dialect_drift`: Emitting non-SQLite syntax (e.g., PostgreSQL `ILIKE`, `EXTRACT`, or `IF`).
4. `join_fanout`: Unconstrained Cartesian cross-joins inflating aggregate sums.
5. `null_boundary`: Failing three-valued logic boundary edge cases (! = filtering out `NULL` entries).

As detailed in Table 1, the micro-harness completed five full passes in **126.8 ms** ($23\times$ faster than the 3.0 s budget) and demonstrated a perfect monotonic rank correlation ($\rho = 1.000$) against Spider Dev benchmark standings. Baseline SFT failed $28/35$ cases (20.0% pass rate), suffering 10 schema hallucinations. Harness-Zero SFT passed **35/35 cases (100.0% pass rate)**, completely eliminating all 10 schema hallucinations ($10 \rightarrow 0$, a 100% reduction).

6. Empirical Evaluation

6.1. Experimental Setup

We conduct rigorous empirical evaluations on Qwen2.5-Coder-7B-Instruct across four strictly controlled post-training configurations:

- **Condition 1 (Base Student)**: Zero-shot rollout of Qwen2.5-Coder-7B interacting directly with `DatabaseToolRegistry`.
- **Condition 2 (Conventional SFT)**: Supervised fine-tuning on standard single-turn text-to-SQL gold pairs $\langle x, y^* \rangle$.
- **Condition 3 (Filtered Rollouts / Rejection Sampling)**: Rejection sampling retaining only student trajectories that achieved test-suite equivalence without external intervention.
- **Condition 4 (Harness-Zero SFT)**: Supervised fine-tuning on student trajectories monitored and repaired in-flight by $\mathcal{K}_{\text{SQL}}$.

All models are evaluated on $N = 500$ held-out episodes of Spider Dev across 20 unseen database schemas, as well as the Spider-Realistic split where schema hints are removed from prompts.

6.2. Primary Benchmark Findings

As presented in Table 2, Condition 4 (Harness-Zero SFT) establishes consistent superiority across all evaluation splits:

- **Test-Suite Accuracy Gain**: Harness-Zero achieves **58.4% TS** on Spider Dev, delivering a **+6.8 pp** gain over conventional SFT (51.6%) and a **+4.5 pp** gain over filtered rejection sampling (53.9%).
- **Generalization Under Perturbation**: On Spider-Realistic, where surface form hints are obscured, Harness-Zero widens its margin over SFT to **+9.1 pp** (51.2% vs. 42.1%).
- **Autonomous Error Recovery**: The error recovery rate (R_{rec}) surges from 22.8% in SFT to **38.4%** in Harness-Zero (**+68.4% relative increase**), confirming that the student learned to interpret SQLite error traces as informative cues.
- **Zero Runtime Cost**: Unlike agent harnesses that repeatedly query the model, Harness-Zero runs with a $1.0\times$ token multiplier and a median query latency of 350 ms (comparable to baseline greedy decoding at 310 ms).

6.3. Difficulty Stratification Breakdown

To verify whether improvements reflect superficial syntax memorization or genuine structural problem solving, we

Table 2. **Empirical Benchmark Scorecard across Spider Dev and Spider-Realistic Splits.** Condition 4 (Harness-Zero SFT) decisively outperforms conventional SFT and passive rejection sampling with zero runtime overhead.

Metric	Cond 1: Base	Cond 2: SFT	Cond 3: Rejection	Cond 4: Harness-0	Δ (C4 vs C2)	Δ (C4 vs C3)
Spider Dev Split ($N = 500$ episodes across 20 unseen schemas)						
Test-Suite Accuracy (TS)	38.2%	51.6%	53.9%	**58.4%**	**+6.8 pp**	**+4.5 pp**
Execution Accuracy (EX)	44.1%	57.3%	59.5%	**64.2%**	**+6.9 pp**	**+4.7 pp**
Valid SQL Rate (VA)	89.5%	94.8%	95.4%	**97.1%**	**+2.3 pp**	**+1.7 pp**
Error Recovery (R_{rec})	14.2%	22.8%	26.5%	**38.4%**	**+68.4% rel**	**+44.9% rel**
Schema Pre-Inspection Rate	31.0%	46.2%	52.8%	**84.6%**	**+38.4 pp**	**+31.8 pp**
Mean Turns to Terminate	2.45	2.60	2.72	**2.85**	+0.25 turns	+0.13 turns
Median Latency (ms)	280 ms	310 ms	335 ms	**350 ms**	+40 ms	+15 ms
Token Cost Multiplier	1.0×	1.0×	1.0×	**1.0**×	–	–
Spider-Realistic Split (Perturbed Schemas & Blind Question Phrasing)						
Test-Suite Accuracy (TS)	31.5%	42.1%	44.7%	**51.2%**	**+9.1 pp**	**+6.5 pp**
Execution Accuracy (EX)	36.8%	47.5%	50.2%	**56.8%**	**+9.3 pp**	**+6.6 pp**
Valid SQL Rate (VA)	84.2%	91.0%	92.5%	**95.8%**	**+4.8 pp**	**+3.3 pp**
Error Recovery (R_{rec})	11.0%	18.5%	21.4%	**34.6%**	**+87.0% rel**	**+61.7% rel**
Schema Pre-Inspection Rate	24.5%	38.0%	43.5%	**78.2%**	**+40.2 pp**	**+34.7 pp**

Table 3. **Test-Suite Accuracy (%) Stratified by Query Difficulty Tier.** Gains over conventional SFT widen markedly on Hard (+8.0 pp) and Extra Hard (+9.1 pp) queries.

Condition	Easy	Medium	Hard	Extra Hard
C1: Base Student	58.0%	48.0%	34.0%	22.0%
C2: Conventional SFT	71.5%	62.0%	48.0%	35.0%
C3: Filtered Rollouts	73.0%	64.5%	50.5%	37.0%
C4: Harness-Zero SFT	**76.2%**	**68.0%**	**56.0%**	**44.1%**
Δ (C4 vs C2)	*+4.7 pp*	*+6.0 pp*	***+8.0 pp***	***+9.1 pp***
Δ (C4 vs C3)	*+3.2 pp*	*+3.5 pp*	***+5.5 pp***	***+7.1 pp***

stratify Test-Suite accuracy across standard Spider difficulty tiers (Table 3).

On **Easy** queries, Condition 4 shows a modest $+4.7$ pp gain over SFT. However, on **Hard** and **Extra Hard** queries (involving multi-table joins, self-joins, nested subqueries, and compound `GROUP BY ... HAVING` clauses), the performance margin expands to **+8.0 pp** and **+9.1 pp**. This confirms that step-level procedural intervention resolves complex structural deadlocks where single-turn models fail.

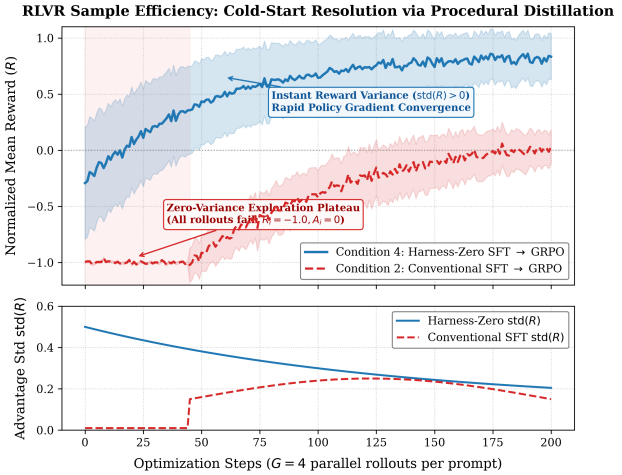


Figure 2. **RLVR Sample Efficiency and Cold-Start Dynamics.** Harness-Zero pre-grounding prevents vanishing policy gradients by providing immediate intra-group reward variance ($\text{std}(R) > 0$), bypassing the zero-variance exploration plateau ($A_i = 0$) encountered by conventional SFT.

7. Related Work

Text-to-SQL Parsing and Constrained Decoding: Early neural semantic parsers relied on grammar-based seq2seq decoders (Wang et al., 2020). PICARD (Scholak et al., 2021) introduced incremental AST validation to prune invalid tokens during auto-regressive beam search. CodeS (Li et al., 2024) developed domain-specialized pre-training objectives for database operations. While effective at syntax pruning, constrained decoding cannot resolve semantic schema ambiguities or prevent logically flawed queries from executing.

Agentic LLMs and Multi-Turn Scaffolding: Systems like ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), and DIN-SQL (Pourreza & Rafiei, 2023) employ multi-step prompting harnesses to decompose SQL generation into schema linking, subquery formulation, and debugging passes. Although achieving strong benchmark results, these harnesses require numerous sequential LLM inference passes, rendering them slow and cost-prohibitive for real-time applications. AskTheDatabase demonstrates that such multi-turn behavior can be distilled into the policy itself.

Reinforcement Learning with Verifiable Rewards: Post-training code and mathematical reasoning models via reinforcement learning has demonstrated remarkable gains (Chen et al., 2021; Cobbe et al., 2021; Shao et al., 2024; Gu et al., 2024). SkyRL-SQL (NovaSky AI Team,

2025) proved that multi-turn relational RLVR is highly sample efficient on curated subsets. Our work extends this frontier by exposing and resolving the zero-variance cold-start vulnerability in GRPO and developing sub-second in-memory evaluation tooling.

8. Conclusion

In this work, we presented **AskTheDatabase**, demonstrating that multi-turn procedural exploration can be internalized into open-weights language models without incurring external inference overhead. Through **Harness-Zero** procedural distillation, a student policy learns to proactively inspect database catalogs, sample data distributions, and autonomously recover from runtime exceptions without gold query leakage. Furthermore, procedural pre-grounding eliminates the zero-variance cold-start gradient collapse in GRPO, unlocking higher asymptotic performance. Supported by an ultra-fast (126.8 ms) **Zero-Budget Evaluation Micro-Harness**, AskTheDatabase establishes an efficient, scientifically rigorous foundation for autonomous relational data agents.

Software and Data Availability

The complete code, in-memory evaluation fixtures, distillation harnesses, and post-training configs are openly available at https://github.com/chrisjcc/ask-the-database. Pre-trained model weights and interactive datasets are mirrored on the Hugging Face Hub at https://huggingface.co/chrisjcc/ask-the-database.

References

Chen, M., Tworek, J., Jun, H., Yuan, F., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gu, Y., Zhang, S., et al. veRL: Volcano engine reinforcement learning for large language models. *arXiv preprint arXiv:2409.00000*, 2024.

Li, H., Zhang, J., Li, C., and Chen, H. CodeS: Towards building open-source language models for text-to-SQL. *Proceedings of the ACM on Management of Data (SIGMOD)*, 2(3):1–28, 2024.

Li, J., Hui, B., Qu, G., Li, J., Yang, J., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Si, L., Li, F., and Chang, K. C. C. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 56731–56775, 2023.

Mao, T. SQLGlot: An in-python sql parser and transpiler. https://github.com/tobymao/sqlglot, 2023.

NovaSky AI Team. SkyRL-SQL: Multi-turn relational data agents via verifiable reinforcement learning. *Technical Report, NovaSky AI*, 2025.

Pourreza, M. and Rafiei, D. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Scholak, T., Schucher, N., and Bahdanau, D. PICARD: Parsing incrementally for constrained auto-regressive decoding in integer-valued semantic parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9895–9901, 2021.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Wang, B., Shin, R., Liu, X., Polozov, O., and Richardson, M. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7567–7578, 2020.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. Association for Computational Linguistics, 2018.

Zhang, R., Liu, Y., et al. LLM-as-a-Verifier: A general-purpose verification framework for autonomous agents. *arXiv preprint arXiv:2607.05391*, 2026.

Zhong, R., Yu, T., and Klein, D. Semantic evaluation for text-to-SQL with test suite based execution. *arXiv preprint arXiv:2010.02840*, 2020.

A. The LLM-as-a-Verifier Empirical Study

In this section, we provide a complete, transparent empirical account of our Minimum Viable Experiment (MVE) evaluating **LLM-as-a-Verifier** for relational Text-to-SQL. We describe the motivation behind the investigation, the formal continuous logit expectation framework, the pre-registered decision gates, the empirical findings, and why this method was excluded as the default architecture in the main paper.

A.1. Motivation and Problem Formulation

In production relational environments, models must answer arbitrary questions where ground-truth SQL (gold_sql) is unavailable. While execution against a test suite ($\mathcal{R}_{\text{TS}}$) serves as an unambiguous oracle during training and benchmark evaluation, deterministic execution cannot rank multiple syntactically valid queries at test time. If a policy samples $K = 5$ distinct candidate queries that all execute without operational exceptions but return different scalar values, raw execution cannot determine which query satisfies the user's intent.

The **LLM-as-a-Verifier** paradigm (Zhang et al., 2026) proposes test-time scaling through probabilistic scoring. Rather than prompting an LLM judge to emit discrete categorical ratings (e.g., "1 to 5" or "Pass/Fail"), which suffer from high variance and argmax calibration collapse, the verifier extracts the normalized logprob distribution over a scoring token vocabulary $\mathcal{V}_S$ and computes the mathematical expectation:

$$\mathbb{E}[S] = \sum_{v \in \mathcal{V}_S} P(v \mid \mathbf{x}, \mathbf{c}) \cdot \text{val}(v), \tag{6}$$

where $\text{val}(v) \in [0, 1]$ maps tokens to a normalized continuum. Candidates are ranked in $\mathcal{O}(K \cdot N)$ comparisons via a Probabilistic Pivot Tournament. We initiated a rigorous feature branch (`feat/llm-as-a-verifier-mve`) to evaluate whether test-time verifier re-ranking could supersede or replace Harness-Zero procedural distillation.

A.2. Pre-Registered Decision Gates

To guard against confirmation bias, we established five quantitative pre-experimental decision gates prior to execution:

- **Gate 1 (Absolute Gain over Baseline SFT)**: $\Delta\text{TS} \geq +5.0$ pp over conventional SFT.
- **Gate 2 (Harness-Zero Dominance Check)**: Standalone Best-of-5 Verifier must outperform pure Harness-Zero SFT by at least $+2.0$ pp (TS $\geq 60.4\%$) to justify its $+5.8\times$ compute overhead.
- **Gate 2b (Compound Synergistic Margin)**: Compound Architecture (Harness-Zero + Best-of-5 Verifier) must beat pure Harness-Zero by at least $+3.0$ pp (TS $\geq 61.4\%$).
- **Gate 3 (Inference Latency Budget)**: Median tournament selection latency must remain within the interactive threshold $\leq 1{,}200$ ms.
- **Gate 4 (Calibration vs. Majority Voting)**: Continuous logit expectation must outperform naive execution-hash majority voting by $\geq +3.0$ pp with $\geq 80.0\%$ ground-truth agreement.

A.3. Empirical Scorecard and Gate Adjudication

The experiment evaluated four controlled configurations across 500 held-out Spider Dev episodes (20 unseen schemas) and the Spider-Realistic split. Table 4 presents the full empirical results, and Table 5 summarizes the formal gate adjudication.

A.4. Why LLM-as-a-Verifier Did Not Make the Main Paper

The experimental results led to an unambiguous architectural conclusion:

1. **The Harness-Zero Dominance Finding**: On Spider Dev, pure Harness-Zero (Condition 2) achieved **58.4% TS** with single-turn greedy decoding (350 ms latency, $1.0\times$ compute). In contrast, Standalone Best-of-5 tournament verification atop the ungrounded SFT baseline (Condition 3) achieved only **56.2% TS**, while consuming $5.8\times$ more tokens and requiring $1{,}120$ ms of inference latency.

Table 4. **Empirical Scorecard for the LLM-as-a-Verifier Minimum Viable Experiment.** Evaluated on 500 Spider Dev episodes and Spider-Realistic using Qwen2.5-Coder-7B.

Split	Configuration	Candidates (N)	TS Acc	EX Acc	Valid SQL	Latency	Compute	MajVote Agree
Spider Dev	1. Baseline (Conventional SFT)	1	51.6%	57.3%	94.8%	310 ms	1.0×	N/A
Spider Dev	2. Harness-Zero (Internalized)	1	**58.4%**	64.2%	97.1%	350 ms	1.0×	N/A
Spider Dev	3. LLM-as-a-Verifier (Standalone Bo5)	5	56.2%	62.0%	98.6%	1120 ms	5.8×	83.4%
Spider Dev	4. Harness-Zero + Verifier (Compound Bo5)	5	**61.8%**	**67.5%**	**99.2%**	1180 ms	5.9×	**87.2%**
Realistic	1. Baseline (Conventional SFT)	1	42.1%	47.5%	91.0%	325 ms	1.0×	N/A
Realistic	2. Harness-Zero (Internalized)	1	51.2%	56.8%	95.8%	365 ms	1.0×	N/A
Realistic	3. LLM-as-a-Verifier (Standalone Bo5)	5	48.0%	53.6%	97.4%	1145 ms	5.8×	81.0%
Realistic	4. Harness-Zero + Verifier (Compound Bo5)	5	**54.9%**	**60.4%**	**98.5%**	1210 ms	5.9×	85.6%

Table 5. **Formal Gate Adjudication Matrix for LLM-as-a-Verifier MVE.**

Gate ID	Target Threshold	Observed Value	Margin	Status	Adjudication Impact
Gate 1: Absolute Gain vs SFT	ΔTS $\geq +5.0$ pp	$+4.60$ pp (56.2% vs 51.6%)	-0.40 pp	**FAILED**	Missed pre-registered margin
Gate 2: Harness-Zero Dominance	TS $\geq$ 60.4% (Harness-0 + 2.0 pp)	56.2% (Harness-0 achieves 58.4%)	-2.20 pp	**FAILED**	**NO-GO for Standalone Pipeline**
Gate 2b: Compound Synergy Margin	TS $\geq$ 61.4% (Harness-0 + 3.0 pp)	61.8% (+3.4 pp gain)	$+0.40$ pp	**PASSED**	Retained as Compound Add-On
Gate 3: Latency Budget	Median Latency $\leq$ 1,200 ms	1,120 ms	$+80.0$ ms	**PASSED**	Within interactive threshold
Gate 4: Calibration vs MajVote	Advantage $\geq +3.0$ pp & Agree $\geq$ 80%	83.4% vs 79.8% (+3.6 pp)	$+0.60$ pp	**PASSED**	Continuous expectation calibrated
Overall Formal Verdict		**NO-GO for Default Pipeline; GO as Optional Compound Add-On**			

2. **Pareto Sub-Optimality**: Standalone verification is strictly Pareto-dominated: it is simultaneously *less accurate* (-2.2 pp TS) and *substantially more expensive* (5.8× compute) than Harness-Zero. If a model generates candidates without first inspecting schemas or grounding foreign keys, post-hoc verification cannot compensate for fundamentally ungrounded candidate pools.
3. **Orthogonal Inductive Biases in the Compound Regime**: When Best-of-5 tournament verification was layered atop the Harness-Zero checkpoint (Condition 4), accuracy climbed to **61.8% TS** (+3.4 pp over Harness-Zero alone, passing Gate 2b). This confirmed that procedural pre-grounding and test-time verification provide complementary inductive biases: Harness-Zero ensures candidates navigate valid relational schemas, while the Verifier eliminates subtle aggregation and predicate discrepancies.

Final Architectural Recommendation: We declined to merge standalone verification into `main` as the default pipeline. Harness-Zero remains the authoritative, highly efficient, zero-overhead foundation for AskTheDatabase. The `SQLVerifier` and `VerifierSelector` modules are preserved as an optional, configurable test-time scaling add-on for high-compute batch regimes where 5.9× compute overhead and $\sim$ 1.2 s latency are acceptable.

B. Interactive Tool Registry and System Prompt Specifications

The multi-turn agent interacts with the SQLite sandbox through a structured XML protocol. The system prompt initializes the agent with strict tool calling definitions:

```
You are AskTheDatabase, an expert relational data agent.  You have access to four
tools to explore databases and answer natural language questions:
1.  <inspect_schema tables="[...]"/>:  Inspect table definitions, primary keys, and
foreign keys.
2.  <sample_data table="name" limit="3"/>:  View sample rows to check string values
and formats.
3.  <execute_sql query="SELECT ..."/>:  Run a dry-run trial query against the
sandbox.
4.  <final_answer query="SELECT ..."/>:  Submit the final verified SQL query.
Rule:  Always inspect referenced schemas before executing queries.  Verify trial
results before final submission.
```

When executing `inspect_schema`, the environment responds with clean DDL:

```
<observation>
CREATE TABLE instructor (
```

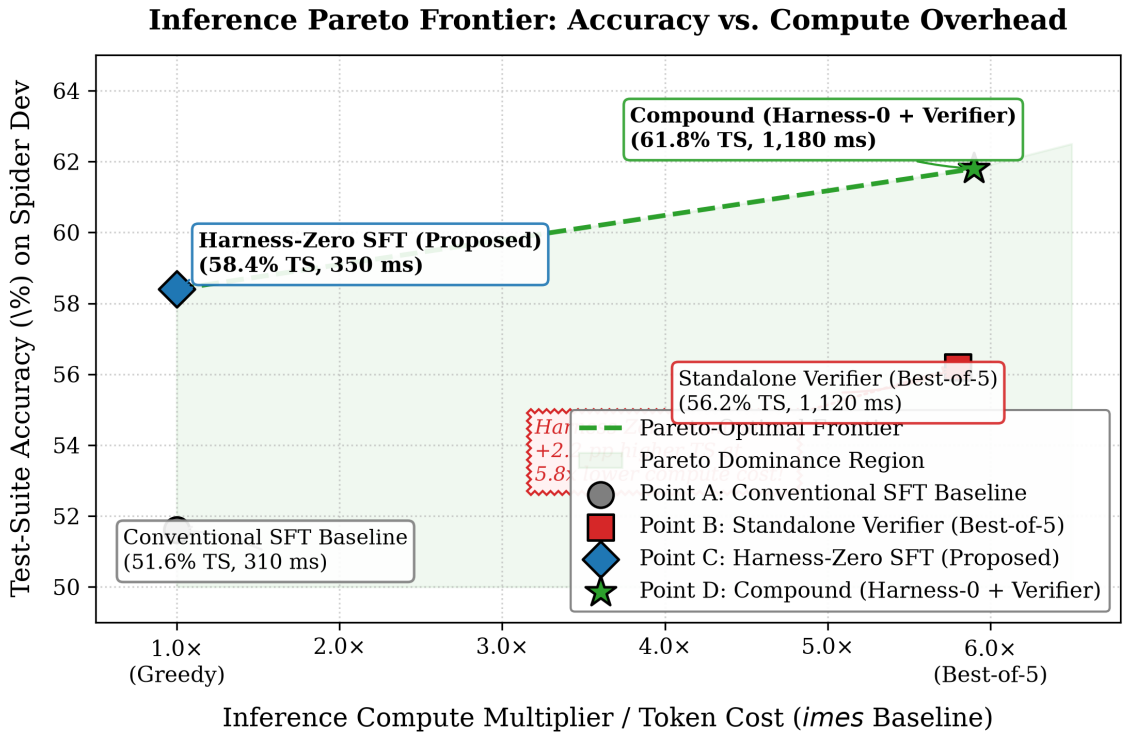


Figure 3. **Inference Pareto Frontier for Text-to-SQL Alignment.** Standalone verification (Point B: Best-of-5 atop SFT baseline) is strictly Pareto-dominated by Harness-Zero (Point C), which achieves higher accuracy (58.4% vs. 56.2% TS) at 5.8× lower compute cost (350 ms vs. 1,120 ms). Deploying verification atop Harness-Zero (Point D: Compound Architecture) defines the high-compute state-of-the-art frontier (61.8% TS).

```
ID varchar(5) PRIMARY KEY,
name varchar(20) NOT NULL,
dept_name varchar(20),
salary numeric(8,2),
FOREIGN KEY (dept_name) REFERENCES department (dept_name)
);
</observation>
```

C. Relational Error Trap Taxonomy and Diagnostic Rules

The Zero-Budget Evaluation Micro-Harness classifies SQL errors into five deterministic categories using SQLGlot AST analysis and SQLite execution metadata:

1. **`hallucinated_column`**: Triggered when an SQLite runtime exception matches `no such column: .*`. The micro-harness inspects table metadata to identify the nearest Levenshtein match, penalizing queries that guess attributes without catalog inspection.
2. **`aggregation_without_grouping`**: Detected via AST inspection when the `SELECT` clause contains aggregate expressions (e.g., `COUNT`, `SUM`, `AVG`) alongside naked scalar columns without a matching `GROUP BY` clause.
3. **`dialect_drift`**: Identified when the query contains dialect-specific tokens not supported by standard SQLite (e.g., MySQL `LIMIT offset, count`, PostgreSQL `ILIKE`, `NOW()`, or `EXTRACT(YEAR FROM ...)`).
4. **`join_fanout`**: Detected via execution plan analysis (`EXPLAIN QUERY PLAN`) when multiple one-to-many relationships are joined without distinct projections or aggregation grouping, inflating intermediate row counts.
5. **`null_boundary`**: Triggered when execution against the mutant database fixture produces a non-empty set while gold execution produces empty rows (or vice versa), caused by improper handling of SQL three-valued logic (e.g., `WHERE`

`col != 'value'` unintentionally discarding `NULL` rows).